



MPPT Solar Charge Controller

Data Parsing Specification

Add-On Data Contract between Firmware & Backend Teams

Document ID	WC-MA-DPC-v1.0
Version	1.0
Date	2026-03-09
Status	Final
Companion	WeatherStation_DataParsingContractRevision.pdf (v1.0)

Contents

1. Overview
2. Data Transmission via MQTT
3. Field Definitions (Tag 0x02 — MPPT, Length 0x1B)
4. Data Parsing Logic
5. Parsing Examples
6. Error Status Reporting (Endpoint 65)
7. Key Differences from Weather Station
8. Firmware Source References
- A. Appendix: Complete Byte Map
- B. Appendix: Backend Pseudocode

1. Overview

This document defines the contract between the Firmware (FW) team responsible for the MPPT solar charge controller add-on and the Backend (BE) team responsible for processing the data. It specifies the MQTT topic structure, payload format using a Tag-Length-Value (TLV) scheme, field definitions for the MPPT data block, and the required parsing logic.

The MPPT add-on reads 7 register groups from an EPEver-compatible solar charge controller over Modbus RTU (RS485) and aggregates them into a single TLV packet transmitted over the Wirepas mesh network.

2. Data Transmission via MQTT

2.1 MQTT Topic Structure

Sensor data is published to the following MQTT topic:

```
<env>/gw-event/received_data/<gw-id>/<sink-id>/<net_id>/61/61
```

Where:

- <env>: The deployment environment. Possible values: production, test, local.
- <gw-id>: The unique serial ID of the gateway device.
- <sink-id>: The unique serial ID of the sink device (also used as the sink's mesh node address).
- <net_id>: The identifier for the Wirepas mesh network.
- 61/61: Static identifiers indicating Source Endpoint 61, Destination Endpoint 61 (Modbus data).

Note: Additional metadata, such as the originating node's source address within the mesh network, is provided within the accompanying Protobuf message structure wrapping the payload. This document focuses specifically on parsing the raw byte payload field within that structure.

2.2 MQTT Payload Format (TLV Structure)

The payload field within the Protobuf message contains one or more data blocks formatted using a Tag-Length-Value (TLV) structure. This allows both MPPT and Weather Station data to share the same endpoint.

Each TLV block has the following format:

- Tag (T): 1 byte. Uniquely identifies the type of data block that follows.
- Length (L): 1 byte. Specifies the length (in bytes) of the Value field only.
- Value (V): L bytes. The actual data payload for the specified Tag.

Current Implementation:

- MPPT Tag: 0x02
- MPPT Value Length: 27 bytes (1-byte bitmask + 26 bytes register data) = 0x1B

Example Payload Structure (MPPT Data Only):

Tag	Length	Value (MPPT Data – 27 bytes)
01	1B	7F 00 00 00 04 00 00 00 64 00 00 00 00 05 14 00 ...

- 01 — Tag indicating Modbus Add-On Data.
- 1B — Length of the Value field (27 bytes in hexadecimal).
- 7F — Bitmask byte (see Section 2.4).
- Remaining 26 bytes: Register data as defined in Section 3.

Future Extensibility: Additional add-ons can be included by appending their respective TLV blocks. The backend should be prepared to parse multiple sequential TLV blocks.

2.3 Discriminating MPPT from Weather Station

MPPT and Weather Station use different Tag values on Endpoint 61. The backend can discriminate by Tag alone:

Device Type	Tag	Length	Hex	Value Structure
Weather Station	0x01	32	0x20	16 × 2-byte signed int16 fields
MPPT Controller	0x02	27	0x1B	1-byte bitmask + 26 bytes data

Parsing rule:

- If Length == 0x20 (32): Parse as Weather Station (see companion document).
- If Length == 0x1B (27): Parse as MPPT (this document).
- Otherwise: Log unknown length and skip.

2.4 Bitmask Field

The first byte of the MPPT Value field is a register success bitmask. Each bit indicates whether the corresponding register group was successfully read.

Bit:	6	5	4	3	2	1	0
	RTC	PV Pwr	PV V/I	Batt	Load	SOC	Status
	0x9013	0x310C	0x3100	V/I	Power	0x311A	0x3200
				0x3110	0x331A		

- Bit N = 1: Register group N responded successfully; data bytes are valid.
- Bit N = 0: Register group N failed (timeout / CRC error); data bytes are zero-filled (0x00).
- Bit 7: Reserved (always 0).

Example: Bitmask 0x7F (binary 0111 1111) = all 7 registers OK.

Example: Bitmask 0x5B (binary 0101 1011) = registers 0, 1, 3, 4, 6 OK; registers 2 and 5 failed.

3. Field Definitions (Tag 0x02 — MPPT, Length 0x1B)

The following table defines the fields within the Value part of the TLV structure when Tag == 0x02 and Length == 0x1B. The first byte is the bitmask (Section 2.4). The remaining 26 bytes contain register data in the order shown.

- Byte Order: Big-Endian (Most Significant Byte first) for all 16-bit values.
- 32-bit Composites: Low word first, high word second (see Section 3.2).

3.1 Register Data Layout

Reg	Bit	Offset	Bytes	Modbus Addr	Func	Description
R0	Bit 0	0–5	6	0x3200	0x04	Battery / Charging / Load Status
R1	Bit 1	6–7	2	0x311A	0x04	Battery SOC
R2	Bit 2	8–11	4	0x331A	0x04	Battery Voltage / Current
R3	Bit 3	12–15	4	0x3110	0x04	Battery Temp / Device Temp
R4	Bit 4	16–19	4	0x3100	0x04	PV Array Voltage / Current
R5	Bit 5	20–23	4	0x310C	0x04	Load Voltage / Current
R6	Bit 6	24–25	2	0x9013	0x03	Real-Time Clock

Note: "Byte Offset" is relative to the 26-byte data portion (after the bitmask). In the full 29-byte packet: absolute offset = 3 + value shown (3 = TAG + LEN + bitmask).

3.2 Detailed Field Definitions

R0: Status Registers (0x3200–0x3202) — 6 bytes

Sub-Field	Offset	Bytes	Data Type	Description
R0.a	0–1	2	Unsigned 16-bit	Battery Status
R0.b	2–3	2	Unsigned 16-bit	Charging Equipment Status
R0.c	4–5	2	Unsigned 16-bit	Load / Discharging Status

R0.a — Battery Status (0x3200) Bit Definitions

Bits	Meaning	Values
3:0	Battery voltage status	0=Normal, 1=Over-V, 2=Under-V, 3=Over discharge, 4=Fault
7:4	Battery temperature status	0=Normal, 1=Over Temp, 2=Low Temp
8	Internal resistance abnormal	0=Normal, 1=Abnormal
15	Wrong identification for rated voltage	0=Correct, 1=Wrong

R0.b — Charging Equipment Status (0x3201) Bit Definitions

Bits	Meaning	Values
0	Running	0=Standby, 1=Running
1	Fault	0=Normal, 1=Fault
3:2	Charging status	0=Not Charging, 1=Float, 2=Boost, 3=Equalize
4	PV input short circuit	0=Normal, 1=Short
7	Load MOSFET short circuit	0=Normal, 1=Short
8	Load short circuit	0=Normal, 1=Short
9	Load over-current	0=Normal, 1=Over-current
10	Input over-current	0=Normal, 1=Over-current
13	Charging MOSFET short circuit	0=Normal, 1=Short
15:14	Input voltage status	0=Normal, 1=No input, 2=Higher input V, 3=Error

R0.c — Discharging Equipment Status (0x3202) Bit Definitions

Bits	Meaning	Values
0	Running / Standby	0=Standby, 1=Running
1	Fault	0=Normal, 1=Fault
4	Output over-voltage	0=Normal, 1=Over-voltage
7	Input over-voltage	0=Normal, 1=Over-voltage
11	Short circuit	0=Normal, 1=Short
13:12	Output power level	0=Light, 1=Moderate, 2=Rated, 3=Overload
15:14	Input voltage status	0=Normal, 1=Low, 2=High, 3=No access

R1: Battery SOC (0x311A) — 2 bytes

Offset	Bytes	Data Type	Unit	Description	Coefficient
6–7	2	Unsigned 16-bit	%	Battery State of Charge	1 (direct)

Value range: 0–100 (percent). No scaling needed.

R2: Battery Voltage and Current (0x331A–0x331B) — 4 bytes

Sub	Offset	Bytes	Type	Unit	Description	Coeff
R2.a	8–9	2	Unsigned 16-bit	V	Battery Voltage	0.01
R2.b	10–11	2	Unsigned 16-bit	A	Battery Current (Low)	0.01

Derived value: Battery Power (W) = Battery Voltage × Battery Current

R3: Battery Temperature and Device Temperature (0x3110–0x3111) — 4 bytes

Sub	Offset	Bytes	Type	Unit	Description	Coeff
R3.a	12–13	2	Unsigned 16-bit	°C	Battery Temperature	0.01
R3.b	14–15	2	Unsigned 16-bit	°C	Device Temperature	0.01

R4: PV Array Voltage and Current (0x3100–0x3101) — 4 bytes

Sub	Offset	Bytes	Type	Unit	Description	Coeff
R4.a	16–17	2	Unsigned 16-bit	V	PV Voltage	0.01
R4.b	18–19	2	Unsigned 16-bit	A	PV Current	0.01

R5: Load Voltage and Current (0x310C–0x310D) — 4 bytes

Sub	Offset	Bytes	Type	Unit	Description	Coeff
R5.a	20–21	2	Unsigned 16-bit	V	Load Voltage	0.01
R5.b	22–23	2	Unsigned 16-bit	A	Load Current	0.01

Derived value: Load Power (W) = Load Voltage × Load Current

R6: Real-Time Clock (0x9013) — 2 bytes

Offset	Bytes	Data Type	Unit	Description
24–25	2	Packed byte	MM:SS	MPPT device clock

RTC Decoding (per EPEver Protocol v2.5, register 0x9013: D15–D8=Minutes, D7–D0=Seconds):

```
high_byte = byte[24]           // Minutes (0-59)
low_byte  = byte[25]           // Seconds (0-59)
time_string = sprintf("%02d:%02d", high_byte, low_byte)
```

4. Data Parsing Logic

The backend must first parse the overall TLV structure of the MQTT payload and then parse the content of recognized Value fields.

1. Parse TLV Blocks: Process the payload sequentially. For each block:
 - Read the Tag (1 byte).
 - Read the Length (1 byte).
 - Read Length bytes for the Value.
 - Verify that Length bytes were successfully read.
2. Identify Block Type: Check the Tag and Length values.
 - If Tag == 0x02 and Length == 0x1B (27): Proceed to Step 3 to parse MPPT data.
 - If Tag == 0x01 and Length == 0x20 (32): Parse as Weather Station (see companion document).
 - If Tag is unknown/unsupported: Log the unknown tag and its length, then skip its Value section.
 - If payload ends prematurely: Handle error appropriately (e.g., log incomplete data).

3. Parse MPPT Value Data (Tag 0x02, Length 0x1B):

Process the 27-byte Value data obtained in Step 1:

- a. Read Bitmask: Extract the first byte as the register success bitmask.
 - b. For each register group R0 through R6:
 - i. Check bitmask: If bit N is 0, skip (treat as NULL / unavailable).
 - ii. Extract bytes at the offset specified in Section 3.1.
 - iii. Combine bytes into 16-bit or 32-bit integers (Big-Endian).
 - iv. Apply scaling: Multiply by the Coefficient specified in Section 3.2.
 - v. Decode bitfields for status registers (R0) per the tables in Section 3.2.
 - vi. Assign units from the Field Definitions table.
4. Repeat: If the end of the payload has not been reached, return to Step 1.

5. Parsing Examples

5.1 Full Successful Packet (All 7 Registers OK)

Raw Hex (29 bytes):

02 1B 7F 00 00 00 04 00 00 00 64 00 00 00 00 05 14 00 64 0D 48 00 32 00 00 00 00 0C 1E

Step-by-step decode:

Field	Hex Bytes	Interpretation
TAG	02	MPPT Modbus Add-On Data
LEN	1B	27 bytes
Bitmask	7F	0111 1111 = all 7 registers OK
R0 Batt Status	00 00	0x0000 = Normal
R0 Chg Status	00 04	0x0004: bits 3:2 = 01 = Float charge mode
R0 Dischg Status	00 00	0x0000 = Standby, Normal
R1 SOC	00 64	100 → 100%
R2 Batt V	00 00	0x0000 = 0 → 0 × 0.01 = 0.00 V
R2 Batt I(L)	00 00	0x0000 = 0 → 0 × 0.01 = 0.00 A
R3 Batt Temp	05 14	0x0514 = 1300 → 1300 × 0.01 = 13.00 °C
R3 Dev Temp	00 64	0x0064 = 100 → 100 × 0.01 = 1.00 °C
R4 PV Voltage	0D 48	0x0D48 = 3400 → 3400 × 0.01 = 34.00 V
R4 PV Current	00 32	0x0032 = 50 → 50 × 0.01 = 0.50 A
R5 Load V	00 00	0x0000 = 0 → 0.00 V
R5 Load I	00 00	0x0000 = 0 → 0.00 A
R6 RTC	0C 1E	Minutes=12, Seconds=30 → 12:30

5.2 Partial Data (Registers 2 and 5 Failed)

Bitmask: 0x5B = binary 0101 1011

Bit	Register	Status	Action
0	R0 Status	1 (OK)	Parse normally
1	R1 SOC	1 (OK)	Parse normally
2	R2 Batt V/I	0 (FAIL)	Skip — zero-filled, treat as NULL
3	R3 Temps	1 (OK)	Parse normally
4	R4 PV V/I	1 (OK)	Parse normally
5	R5 Load V/I	0 (FAIL)	Skip — treat as NULL
6	R6 RTC	1 (OK)	Parse normally

5.3 Decoding a 16-bit Value (Battery Temperature)

Applies to R3.a bytes at data offset 12–13.

- Hexadecimal Bytes: 05 14
- Combined 16-bit Value (Big-Endian): 0x0514
- As Unsigned Integer: 1300
- Apply Coefficient (0.01): 1300 × 0.01 = 13.00
- Result: 13.00 °C (Battery Temperature)

5.4 Decoding a 16-bit Value (Load Voltage)

Applies to R5.a bytes at data offset 20–21.

- Hexadecimal Bytes: 06 A4
- Combined 16-bit Value (Big-Endian): 0x06A4 = 1700
- Apply Coefficient (0.01): $1700 \times 0.01 = 17.00$
- Result: 17.00 V (Load Voltage)

5.5 Decoding RTC Time

Applies to R6 bytes at data offset 24–25.

- Hexadecimal Bytes: 0E 2D
- High Byte (Minutes): 0x0E = 14
- Low Byte (Seconds): 0x2D = 45
- Result: 14:45 (14 minutes, 45 seconds past the hour)

Special Case: Unavailable Data

Unlike the Weather Station (which uses 0x7FFF as a sentinel for disconnected sensors), the MPPT uses a bitmask to indicate unavailable data. If bitmask bit N = 0, the corresponding register's data bytes are zero-filled. The backend should treat these fields as NULL/unavailable — NOT as a reading of zero.

6. Error Status Reporting (Endpoint 65)

In addition to data on Endpoint 61, the firmware sends error status reports on a separate endpoint.

6.1 Error Status MQTT Topic

```
<env>/gw-event/received_data/<gw-id>/<sink-id>/<net_id>/65/65
```

6.2 Error Status Payload

The error status payload is a fixed 3-byte structure (no TLV wrapper):

Byte	Bytes	Description	Values
0	1	LED State	0=Normal, 1=Problem, 2=Critical
1	1	Recent Timeout Count	0–255 (resets after each report)
2	1	Recent CRC Error Count	0–255 (resets after each report)

LED State Interpretation:

- 0 (Normal): Device is communicating normally.
- 1 (Problem): Response timeouts detected, but device is still functional.
- 2 (Critical): Hardware initialization failure or persistent communication errors.

7. Key Differences from Weather Station

This section summarizes the differences between the MPPT and Weather Station parsing contracts for backend developers who support both.

Aspect	Weather Station	MPPT
Tag	0x01	0x02
Length	0x20 (32 bytes)	0x1B (27 bytes)
Discriminator	Length == 32	Length == 27
Bitmask	None	1st byte of Value
Data fields	16 uniform 2-byte fields	7 variable-size groups (2–6 B)
Data type	Signed 16-bit (Two's Comp.)	Unsigned 16-bit and 32-bit
Scaling	Per-field (0.1, 0.001, 1)	0.01 for V/A/W, 1 for SOC
Unavailable	0x7FFF sentinel	Bitmask bit=0, zero-filled
Status fields	None (all measurements)	RO has bitfield status regs
Channel remap	Yes (4 remap configs)	No (fixed register order)

8. Firmware Source References

For implementation verification, these are the authoritative source files:

File	Purpose
wc_addon_modbus_mppt.c	MPPT polling, TLV aggregation, packet construction
wc_addon_modbus_mppt.h	Register config structures, constants
wc_addon_modbus.c	Shared: CRC, auto-detect, LED, errors, data push
wc_addon_modbus.h	Shared constants, endpoint defs, device type enum
wakecap_config.h	Endpoint numbers (EP 61 data, EP 65 error)
tools/mppt_parse.py	Reference Python parser for MPPT TLV packets

A. Appendix: Complete Byte Map

Complete byte-by-byte map of the 29-byte MPPT TLV packet:

Byte	Name	Field	Type	Notes
[0]	TAG	Tag	uint8	Always 0x02
[1]	LEN	Length	uint8	Always 0x1B (27)
[2]	BM	Bitmask	uint8	Bit N = reg N success
[3–4]	D+0,1	Battery Status	uint16 BE	Bitfield (R0.a)
[5–6]	D+2,3	Charging Status	uint16 BE	Bitfield (R0.b)
[7–8]	D+4,5	Dischg. Status	uint16 BE	Bitfield (R0.c)
[9–10]	D+6,7	SOC	uint16 BE	0–100 %
[11–12]	D+8,9	Battery Voltage	uint16 BE	× 0.01 V (0x331A)
[13–14]	D+10,11	Battery Current L	uint16 BE	× 0.01 A (0x331B)
[15–16]	D+12,13	Battery Temp	uint16 BE	× 0.01 °C (0x3110)
[17–18]	D+14,15	Device Temp	uint16 BE	× 0.01 °C (0x3111)
[19–20]	D+16,17	PV Voltage	uint16 BE	× 0.01 V (0x3100)
[21–22]	D+18,19	PV Current	uint16 BE	× 0.01 A (0x3101)
[23–24]	D+20,21	Load Voltage	uint16 BE	× 0.01 V (0x310C)
[25–26]	D+22,23	Load Current	uint16 BE	× 0.01 A (0x310D)
[27]	D+24	RTC Minutes	uint8	0–59 (0x9013 D15:D8)
[28]	D+25	RTC Seconds	uint8	0–59 (0x9013 D7:D0)

B. Appendix: Backend Pseudocode (Python)

TLV Router

```
def parse_modbus_payload(payload: bytes):
    """Parse one or more TLV blocks from an EP 61 MQTT payload."""
    offset, results = 0, []
    while offset < len(payload):
        if offset + 2 > len(payload):
            log_error("Incomplete TLV header"); break
        tag, length = payload[offset], payload[offset + 1]
        offset += 2
        if offset + length > len(payload):
            log_error("TLV value truncated"); break
        value = payload[offset : offset + length]
        offset += length
        if tag == 0x01 and length == 0x20:
            results.append(parse_weather_station(value))
        elif tag == 0x02 and length == 0x1B:
            results.append(parse_mppt(value))
        else:
            log_warning(f"Unknown TLV: tag=0x{tag:02X}, len={length}")
    return results
```

MPPT Parser

```
def parse_mppt(value: bytes) -> dict:
    """Parse 27-byte MPPT TLV value into structured data."""
    bitmask, data = value[0], value[1:]    # 26 bytes
    result = {"bitmask": bitmask}

    def u16(off): return (data[off] << 8) | data[off + 1]
    def u32c(off): return u16(off) | (u16(off + 2) << 16)

    if bitmask & 0x01:                        # R0: Status
        result["battery_status"] = u16(0)
        result["charging_status"] = u16(2)
        result["discharging_status"] = u16(4)
    if bitmask & 0x02:                        # R1: SOC
        result["soc_percent"] = u16(6)
    if bitmask & 0x04:                        # R2: Batt V/I
        result["battery_voltage_v"] = u16(8) * 0.01
        result["battery_current_a"] = u16(10) * 0.01
    if bitmask & 0x08:                        # R3: Temps
        result["battery_temp_c"] = u16(12) * 0.01
        result["device_temp_c"] = u16(14) * 0.01
    if bitmask & 0x10:                        # R4: PV V/I
        result["pv_voltage_v"] = u16(16) * 0.01
        result["pv_current_a"] = u16(18) * 0.01
    if bitmask & 0x20:                        # R5: Load V/I
        result["load_voltage_v"] = u16(20) * 0.01
        result["load_current_a"] = u16(22) * 0.01
    if bitmask & 0x40:                        # R6: RTC
        result["rtc_minutes"] = data[24]
        result["rtc_seconds"] = data[25]
    return result
```